Développer pour le Cloud

INFO - M2 Expert en développement web

Fondamentaux Cloud

Fondamentaux Cloud

1. Le Cloud – idée centrale
2. Matériel → OS → Runtime → Code
3. IaaS / PaaS / SaaS – logique de responsabilité
4. Docker – rôle réel
5. PaaS (code) vs PaaS (containers)
6. Cas d’usage typiques par modèle cloud
7. Élasticité des modèles cloud

1. Le cloud - Définition pédagogique

Le cloud est un modèle de mise à disposition de ressources informatiques à la demande, facturées à l'usage, gérées par un fournisseur, permettant de se concentrer sur l'application plutôt que sur l'infrastructure.

1. Le cloud - Définition pédagogique

Le cloud est un modèle de mise à disposition de ressources informatiques à la demande, facturées à l'usage, gérées par un fournisseur, permettant de se concentrer sur l'application plutôt que sur l'infrastructure.

- Le cloud ne supprime pas les serveurs

1. Le cloud - Définition pédagogique

Le cloud est un modèle de mise à disposition de ressources informatiques à la demande, facturées à l'usage, gérées par un fournisseur, permettant de se concentrer sur l'application plutôt que sur l'infrastructure.

- Le cloud ne supprime pas les serveurs
- Il supprime l'obligation de deviner l'avenir

1. Le cloud - Définition pédagogique

Le cloud est un modèle de mise à disposition de ressources informatiques à la demande, facturées à l'usage, gérées par un fournisseur, permettant de se concentrer sur l'application plutôt que sur l'infrastructure.

- Le cloud ne supprime pas les serveurs
- Il supprime l'obligation de deviner l'avenir
- Il introduit une logique élastique (adaptation continue)

1. Le cloud - Opposition utile

Infrastructure classique

Cloud

1. Le cloud - Opposition utile

Infrastructure classique

- dimensionnement rigide

Cloud

1. Le cloud - Opposition utile

Infrastructure classique

- dimensionnement rigide
- anticipation incertaine

Cloud

1. Le cloud - Opposition utile

Infrastructure classique

- dimensionnement rigide
- anticipation incertaine

Cloud

- ajustement progressif à la charge réelle

2. Matériel → OS → Runtime

2. Matériel (Hardware)

- Fournit les ressources physiques : CPU, mémoire, disque, carte réseau
- N’exécute que des instructions très bas niveau.

2. Système d’exploitation (OS)

- Fait le lien entre matériel et programmes
- Gère CPU, mémoire, fichiers, réseau
- Exécute des programmes

2. Runtime

Définition : un **programme** spécialisé qui exécute du code écrit dans un langage donné.

- exécuté par l'OS
- utilise les ressources fournies par l'OS

Exemples :

2. Runtime

Définition : un **programme** spécialisé qui exécute du code écrit dans un langage donné.

- exécuté par l'OS
- utilise les ressources fournies par l'OS

Exemples :

- Node.js → JavaScript
- Python → Python,
- JVM → Java

2. Matériel → OS → Runtime

Code

↓

Runtime (Node, Python, JVM…)

↓

OS (Linux, Windows…)

↓

Matériel (CPU, RAM, disque, réseau)

3. IaaS / PaaS / SaaS

Logique de responsabilité

La question à se poser :

- Qui est responsable de quoi ?

3. IaaS – Infrastructure as a Service

- Le cloud fournit une machine vierge
- A toi de faire tout le reste…

3. IaaS – Infrastructure as a Service

On remplace le serveur physique que l’entreprise possédait par un serveur virtuel.

3. IaaS – Infrastructure as a Service

On remplace le serveur physique que l'entreprise possédait par un serveur virtuel.

👉 Le serveur virtuel est une abstraction logicielle.

3. IaaS – Infrastructure as a Service

On remplace le serveur physique que l'entreprise possédait par un serveur virtuel.

👉 Le serveur virtuel est une abstraction logicielle.

👉 Le matériel physique existe toujours, mais il est dans les datacenters du fournisseur cloud.

3. IaaS – Infrastructure as a Service

On remplace le serveur physique que l'entreprise possédait par un serveur virtuel.

👉 Le serveur virtuel est une abstraction logicielle.

👉 Le matériel physique existe toujours, mais il est dans les datacenters du fournisseur cloud.

Avantages : tu ne gères plus le matériel (pannes, remplacement, électricité, refroidissement)

3. IaaS – Infrastructure as a Service

Qui est responsable de quoi (toi ou le cloud) ?

- Code ←
- Runtime ←
- OS ←
- VM ←
- Hardware ←

3. IaaS – Infrastructure as a Service

Qui est responsable de quoi ?

- Code ← toi
- Runtime ← toi
- OS ← toi
- VM ← cloud
- Hardware ← cloud

3. PaaS – Platform as a Service

- Je donne mon code
- La plateforme l’exécute

3. PaaS – Platform as a Service

Qui est responsable de quoi (toi ou le cloud) ?

- Code ←
- Runtime ←
- OS ←
- Infrastructure ←

3. PaaS – Platform as a Service

Qui est responsable de quoi (toi ou le cloud) ?

- Code ← toi
- Runtime ← cloud
- OS ← cloud
- Infrastructure ← cloud

3. SaaS – Software as a Service

- Avec un SaaS, je ne déploie rien.
- J’utilise.

3. SaaS – Software as a Service

Qui est responsable de quoi (toi ou le cloud) ?

- Application complète ←

3. SaaS – Software as a Service

Qui est responsable de quoi (toi ou le cloud) ?

- Application complète ← cloud

4. Docker – rôle réel

- L’OS exécute Docker
- Docker exécute des containers

4. Docker – rôle réel

- L'OS exécute Docker
- Docker exécute des containers

👉 Docker ne virtualise pas un OS complet (contrairement à une VM).

4. Docker – rôle réel

- L'OS exécute Docker
- Docker exécute des containers

👉 Docker ne virtualise pas un OS complet (contrairement à une VM).

- Une VM virtualise une machine.
- Un container isole un processus.

4. Docker – rôle réel

- Réduit fortement les écarts entre environnements

4. Docker – rôle réel

- Réduit fortement les écarts entre environnements

Docker garantit

Docker NE garantit PAS

4. Docker – rôle réel

- Réduit fortement les écarts entre environnements

Docker garantit

- Même runtime
- Même dépendances

Docker NE garantit PAS

- Réseau
- Charge réelle
- Sécurité

4. Docker – rôle réel

👉 Docker standardise l’environnement, pas l’infrastructure.

5. PaaS (code) vs PaaS (containers)

5. PaaS orienté code

👉 Le code est fourni, la plateforme décide comment l’exécuter

5. PaaS orienté code

- 👉 Le code est fourni, la plateforme décide comment l'exécuter
- 👉 Peu de contrôle sur l'environnement

5. PaaS orienté code

👉 Le code est fourni, la plateforme décide comment l'exécuter

👉 Peu de contrôle sur l'environnement

Exemples :

- Heroku
- Azure App Service (mode standard)

5. PaaS orienté containers

👉 L’image docker est fournie, la plateforme exécute le container

5. PaaS orienté containers

- 👉 L'image docker est fournie, la plateforme exécute le container
- 👉 Environnement maîtrisé, portabilité élevée

5. PaaS orienté containers

👉 L'image docker est fournie, la plateforme exécute le container

👉 Environnement maîtrisé, portabilité élevée

Exemples:

- Google cloud run
- Azure Container Apps

5. Portabilité ?

Capacité de déplacer une application d'un fournisseur cloud à un autre avec peu ou pas de modifications

Pourquoi la portabilité est recherchée ?

Pourquoi Docker joue un rôle clé ?

5. Portabilité ?

Capacité de déplacer une application d'un fournisseur cloud à un autre avec peu ou pas de modifications

Pourquoi la portabilité est recherchée ?

- éviter la dépendance avec un fournisseur (**vendor lock-in)**
- raisons économiques (coûts, négociation)
- contraintes réglementaires ou géographiques

Pourquoi Docker joue un rôle clé ?

- il standardise l'environnement
- il réduit les dépendances spécifiques à une plateforme

6. Cas d’usage typiques par modèle cloud

6. IaaS – Cas d’usage

- Migration d’une application existante (legacy)
- Besoin de contrôle total sur l’OS
- Contraintes spécifiques (sécurité, réseau, dépendances système)

6. IaaS – Cas d’usage

- Migration d’une application existante (legacy)
- Besoin de contrôle total sur l’OS
- Contraintes spécifiques (sécurité, réseau, dépendances système)

Profil d’équipe 👪

- Équipe technique expérimentée
- Capacité à administrer des serveurs

6. PaaS (code) – Cas d’usage

- Application web classique
- Priorité à la vitesse de livraison
- Peu de contraintes techniques spécifiques

6. PaaS (code) – Cas d’usage

- Application web classique
- Priorité à la vitesse de livraison
- Peu de contraintes techniques spécifiques

Profil d’équipe 👪

- Équipe orientée développement
- Peu ou pas de compétences système

6. PaaS (containers) – Cas d’usage

- Besoin de contrôler l’environnement d’exécution
- Plusieurs environnements identiques (dev, staging, prod)
- Portabilité entre cloud providers

6. PaaS (containers) – Cas d’usage

- Besoin de contrôler l’environnement d’exécution
- Plusieurs environnements identiques (dev, staging, prod)
- Portabilité entre cloud providers

Profil d’équipe 👪

- Équipe DevOps ou tech lead
- Maîtrise de Docker

6. SaaS – Cas d’usage

- Besoin fonctionnel standard
- Aucun avantage à développer en interne

6. SaaS – Cas d’usage

- Besoin fonctionnel standard
- Aucun avantage à développer en interne

Profil d’équipe 👪

- Tous profils

7. Élasticité ?

7. Élasticité ?

👉 L'élasticité est la capacité d'un système à **adapter dynamiquement ses ressources** à la charge.

Comment, en jouant sur quel paramètres ?

7. Élasticité ?

👉 L'élasticité est la capacité d'un système à **adapter dynamiquement ses ressources** à la charge.

Comment, en jouant sur quel paramètres ?

- le nombre d'instances
- la puissance de calcul
- la capacité de stockage

7. Élasticité et modèles cloud

Élasticité + IaaS ?

Élasticité + PaaS

Élasticité + SaaS

7. Élasticité et modèles cloud

Élasticité + IaaS : 👍 Possible, mais nécessite configuration

Élasticité + PaaS ?

Élasticité + SaaS

7. Élasticité et modèles cloud

Élasticité + IaaS : 👍 Possible, mais nécessite configuration

Élasticité + PaaS : 👍 Native

Élasticité + SaaS ?

7. Élasticité et modèles cloud

Élasticité + IaaS : 👍 Possible, mais nécessite configuration

Élasticité + PaaS : 👍 Native

Élasticité + SaaS : 👍 Oui, mais invisible pour l'utilisateur

7. Modèles cloud

⚠️ **Point clé :**

Les modèles cloud (IaaS / PaaS / SaaS) sont indépendants des capacités du cloud (élasticité, haute disponibilité, facturation à l'usage).

Docker enseignable

Docker enseignable

1. Qu’est-ce que Docker ?
2. Docker et Virtualisation
3. Image vs Container
4. Stateless
5. Le Dockerfile
6. Cycles Docker
7. Construire une image
8. Démarrer un container
9. ENTRYPOINT, commande principale

1. Qu’est-ce que Docker ?

Docker est un outil qui permet d’exécuter une application dans un **environnement isolé et reproductible.**

On apelle cet environement : un **container**

2. Container et VM ?

Est-ce que Docker est un outils de virtualisation de machine ?

2. Container et VM ?

⚠️ Docker n’est **PAS** un outil de virtualisation de machines.

2. Container et VM ?

⚠️ Docker n'est **PAS** un outil de virtualisation de machines.

Machine virtuelle (VM)

- Virtualise une machine entière
- Contient un OS complet
- Lourde
- Démarrage lent

≠

Container

2. Container et VM ?

⚠️ Docker n'est **PAS** un outil de virtualisation de machines.

Machine virtuelle (VM)

- Virtualise une machine entière
- Contient un OS complet
- Lourde
- Démarrage lent

≠

Container

- Isole un processus
- Partage le noyau de l'OS hôte
- Léger
- Démarrage rapide

3. Image vs Container

Image

- Modèle figé
- Construite une fois
- Immuable
- Résultat d’un Dockerfile

3. Image vs Container

Image

- Modèle figé
- Construite une fois
- Immuable
- Résultat d'un Dockerfile

Container

- Instance vivante d'une image
- Démarre et s'arrête
- Éphémère
- Exécute une commande

3. Image vs Container

Image

- Modèle figé
- Construite une fois
- Immuable
- Résultat d'un Dockerfile

Container

- Instance vivante d'une image
- Démarre et s'arrête
- Éphémère
- Exécute une commande

Image = plan
Container = exécution

3. Image vs Container

Image = plan
Container = exécution

Voir pour trouver un GIF sinon supprimer la slide

4. Stateless

Une notion clé !

Voir pour trouver un GIF

4. Stateless - Pourquoi on en parle ?

Une notion clé !

- les containers sont **éphémères**
- ils peuvent être arrêtés, supprimés, recréés à tout moment

Le cloud fonctionne par **remplacement**, pas par réparation

4. Stateless - Pourquoi on en parle ?

Une notion clé !

- les containers sont **éphémères**
- ils peuvent être arrêtés, supprimés, recréés à tout moment

Le cloud fonctionne par **remplacement**, pas par réparation

👉 **On ne doit pas dépendre de la mémoire locale du container.**

4. Stateless

Une application *stateless* ne conserve **aucune donnée durable** dans le container.

4. Stateless

Une application *stateless* ne conserve **aucune donnée durable** dans le container.

Où stocker l'état (fichiers utilisateurs, sessions, données métier, …) ?

- base de données
- cache externe (Redis)
- stockage cloud

5. Le Dockerfile

Le Dockerfile est un **fichier texte** qui décrit comment **construire une image Docker**.

Exemple complet de Dockerfile :

```
FROM node:18
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
CMD ["npm", "start"]
```

5. Le Dockerfile - FROM

```
FROM node:18
```

node:18 est une image qui contient :

- OS minimal
- runtime

Docker empile des couches à partir de cette image

5. Le Dockerfile - WORKDIR

```
WORKDIR /app
```

Définit l'espace de travail dans le container. Le nom du dossier est **un choix**.

/app est le dossier :

- de travail dans l'image
- à partir duquel s'éxécutent toutes les commandes

5. Le Dockerfile - COPY

```
COPY package.json package-lock.json ./
```

Copier des fichiers de la **machine hôte** vers l'image

6. Cycles Docker : construction vs exécution

- Phase 1 – Construction de l'image à partir d'un Dockerfile

```
docker build
```

6. Cycles Docker : construction vs exécution

- Phase 1 – Construction de l'image à partir d'un Dockerfile

```
docker build
```

- Phase 2 – Démarrage du container à partir d'une image

```
docker run
```

6. Cycles Docker : construction vs exécution

- Phase 1 – Construction de l'image à partir d'un Dockerfile

```
docker build
```

- Phase 2 – Démarrage du container à partir d'une image

```
docker run
```

Ces commandes sont utilisées :

- par les développeurs
- par les pipelines CI/CD

7. Construire l’image - RUN

```
RUN npm install
```

Exécute des commandes **pendant la construction**

Exemples concrets de RUN :

- installer des dépendances système
- installer des dépendances applicatives
- compiler du code

7. Construire l'image - RUN (Problème)

```
RUN apt-get update
RUN apt-get install -y nodejs
RUN npm install
```

Chaque RUN crée une couche. On regroupe quand c'est logique.

Regroupement recommandé :

```
RUN apt-get update \
    && apt-get install -y nodejs \
    && npm install
```

7. Construire l’image - RUN et COPY

```
COPY package.json package-lock.json ./
RUN npm install
COPY . .
```

Pourquoi copier en deux temps ?

7. Construire l'image - RUN et COPY

```
COPY package.json package-lock.json ./
RUN npm install
COPY . .
```

Pourquoi copier en deux temps ?

Docker utilise un système de cache :

- les dépendances changent peu
- le code change souvent

👉 Copier d'abord les dépendances permet de réutiliser les couches existantes

8. Démarrer le container - CMD

```
CMD ["npm", "start"]
```

- Définit la commande exécutée **au démarrage du container**
- Ne modifie pas l'image

8. Démarrer le container - CMD

```
CMD ["npm", "start"]
```

- Définit la commande exécutée **au démarrage du container**
- Ne modifie pas l'image

⚠️ Un container n'exécute **qu'un seul process principal.**

```
CMD ["echo", "hello"]
CMD ["npm", "start"]
```

👉 Seule la dernière instruction CMD est prise en compte.

9. ENTRYPOINT : commande principale

Définit la commande fixe du container

```
ENTRYPOINT ["npm"]
CMD ["start"]
```

ENTRYPOINT = programme
CMD = arguments par défaut

9. ENTRYPOINT : commande principale

Définit la commande fixe du container

```
ENTRYPOINT ["npm"]
CMD ["start"]
```

ENTRYPOINT = programme
CMD = arguments par défaut

👉 Les arguments fournis remplacent CMD.

Si je fais `docker run mon-image` Docker exécutera `npm start`

Si je fais `docker run mon-image test` Docker exécutera `npm test`

9. ENTRYPOINT : commande principale

⚠️ ENTRYPOINT sans CMD : conséquence

```
ENTRYPOINT ["npm"]
```

Si je fais `docker run mon-image` le container démarre sans argument.

👉 Cela provoque souvent une erreur ou un comportement incomplet.

Ajouter un gif catastrophe, explosion

Docker enseignable

- Docker ≠ VM
- Image ≠ container
- Une image → plusieurs containers possibles
- RUN construit l'image
- CMD démarre l'application
- ENTRYPOINT fixe la commande
- Stateless = container jetable

Récap express

Docker Compose

Docker Compose

1. Définifition
2. docker-compose.yml
3. Volumes
4. Variables d’environnement
5. Commandes Docker Compose
6. Et en production Cloud ?
7. L’orchestrateur Cloud

1. Pourquoi Docker Compose existe ?

Une application réelle, c’est souvent :

- une API
- une base de données
- parfois un front
- parfois un cache

👉 Lancer tout ça à la main devient ingérable.

1. Pourquoi Docker Compose existe ?

Une application réelle, c'est souvent :

- une API
- une base de données
- parfois un front
- parfois un cache

👉 Lancer tout ça à la main devient ingérable.

Docker Compose intervient pour résoudre ce problème

1. Qu’est-ce que Docker Compose ?

Docker Compose est un outil qui permet de :

- décrire plusieurs services
- les configurer
- les lancer ensemble

👉 Le tout à partir **d’un seul fichier** : docker-compose.yml

1. Compose ≠ production cloud

Docker Compose **n'est PAS** un orchestrateur cloud.

Utiliser un GIF de grande déception

1. Compose ≠ production cloud

Docker Compose **n'est PAS** un orchestrateur cloud.

Docker Compose est utilisé pour :

- le développement
- les tests
- les environnements locaux

2. docker-compose.yml - Structure minimale

```
services:
  service-name:
    image: image-name
```

- `services` : liste des containers
- chaque service = un container

2. docker-compose.yml - Exemple complet

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD:
password
      POSTGRES_DB: appdb
```

Service api

- `build: .` → utilise le Dockerfile
- `ports` → expose l'API vers l'extérieur
- `depends_on` → dépend du service db

2. docker-compose.yml - Exemple complet

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD:
password
      POSTGRES_DB: appdb
```

Service db

- image existante
- aucune construction
- configuration via variables d'environnement

2. docker-compose.yml - Réseau entre services

Dans Docker Compose :

- chaque service a un **nom**
- ce nom devient un hostname réseau

`<nom_du_service>:<port>`

Exemple :

- service : `db`
- port PostgreSQL : `5432`

👉 L'API se connecte à : `db:5432`

3. Volumes - Stateless, rappel

Les containers sont éphémères, on sépare les responsabilité :

- l'API : stateless
- la base de données : stateful

👉 C'est un principe fondamental du cloud.

3. Volumes - Définition

Qu’est-ce qu’un volume ?

- Un volume est un **espace de stockage persistant**, indépendant du cycle de vie d’un container.

Pourquoi en a-t-on besoin ?

- Sans volume : supprimer un container = perdre les données
- Avec un volume : les données survivent aux redémarrages

3. Volumes - Définition

Qu'est-ce qu'un volume ?

- Un volume est un **espace de stockage persistant**, indépendant du cycle de vie d'un container.

Pourquoi en a-t-on besoin ?

- Sans volume : supprimer un container = perdre les données
- Avec un volume : les données survivent aux redémarrages

Ce qu'on met dedans :

- données de base de données
- fichiers générés (logs, uploads)

3. Volumes - Définition

Qu'est-ce qu'un volume ?

- Un volume est un **espace de stockage persistant**, indépendant du cycle de vie d'un container.

Pourquoi en a-t-on besoin ?

- Sans volume : supprimer un container = perdre les données
- Avec un volume : les données survivent aux redémarrages

Ce qu'on met dedans :

- données de base de données
- fichiers générés (logs, uploads)

⚠️ **Ce qu'on ne met pas** :

- code applicatif
- dépendances

3. Volumes - docker-compose.yml

```yaml
services:
  api:
    ...

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: appdb
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

3. Volumes - docker-compose.yml

```
volumes :
  - db-data:/var/lib/postgresql/data
```

- `db-data`: nom du volume Docker (côté hôte)
- `/var/lib/postgresql/data`: dossier utilisé par PostgreSQL dans le container

👉 **Docker fait le lien**

4. Variables d’environnement

La configuration dépend de l’environnement, elle ne doit pas être codée en dur.

Cela garantit que l’image Docker reste identique.

4. Variables d'environnement

La configuration dépend de l'environnement, elle ne doit pas être codée en dur.

Cela garantit que l'image Docker reste identique.

Ce qu'on met en variables :

- environnement (`NODE_ENV`)
- credentials
- URLs de services externes

⚠️ **Ce qu'on ne met pas :**

- logique métier
- code applicatif

4. Variables d’environnement - docker-compose.yml

```yaml
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: development
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: appdb
      DB_USER: user
      DB_PASSWORD: password
```

5. Commandes Docker Compose

`docker compose up`

- construit les images
- démarre tous les services

`docker compose down`

- arrête et supprime les containers

`docker compose up --build`

- force la reconstruction des images

6. Et en production cloud ?

Docker Compose **peut techniquement être utilisé sur un serveur**. Mais il n'offre **aucune garantie cloud native.**

6. Limites - Pas de haute disponibilité (HA)

- Tous les containers tournent sur une seule machine
- Si le serveur tombe :
  - tous les containers tombent
  - l’application est totalement indisponible

6. Limites - Pas d’auto-scaling

- Docker Compose ne surveille pas la charge
- Il ne sait pas :
  - créer automatiquement de nouveaux containers
  - en supprimer quand la charge baisse

6. Limites - Pas de tolérance réelle aux pannes

- Si un container plante :
  - il n’est pas redémarré intelligemment
- Si la machine tombe :
  - rien ne prend le relais

7. L’orchestrateur cloud

Un orchestrateur cloud est capable de :

- redémarrer automatiquement des containers en panne
- répartir la charge entre plusieurs instances
- scaler dynamiquement selon la charge
- déployer sur plusieurs machines physiques

Dans le cloud, l’orchestration est assurée par des outils comme :

- Kubernetes (standard de fait)
- services managés basés sur Kubernetes (EKS, GKE, AKS, etc.)

Docker Compose

- Docker = un container
- Docker Compose = plusieurs containers
- Communication par nom de service
- Volumes = données persistantes
- Scale = plusieurs instances
- Kubernetes = orchestration cloud

Récap express